

Engenharia de Software III

Bem vindos à Engenharia de
Software III

Profa. Renata Neves

Engenharia de Software III

- ✓ Arquitetura Big-Data;
- ✓ Estilo de arquitetura CQRS;
- ✓ CQRS com o padrão de Evento de Fornecimento (padrão Event Sourcing), ou padrão de terceirização de eventos.

By Microsoft

<https://docs.microsoft.com/pt-br/azure/architecture/>



Engenharia de Software III

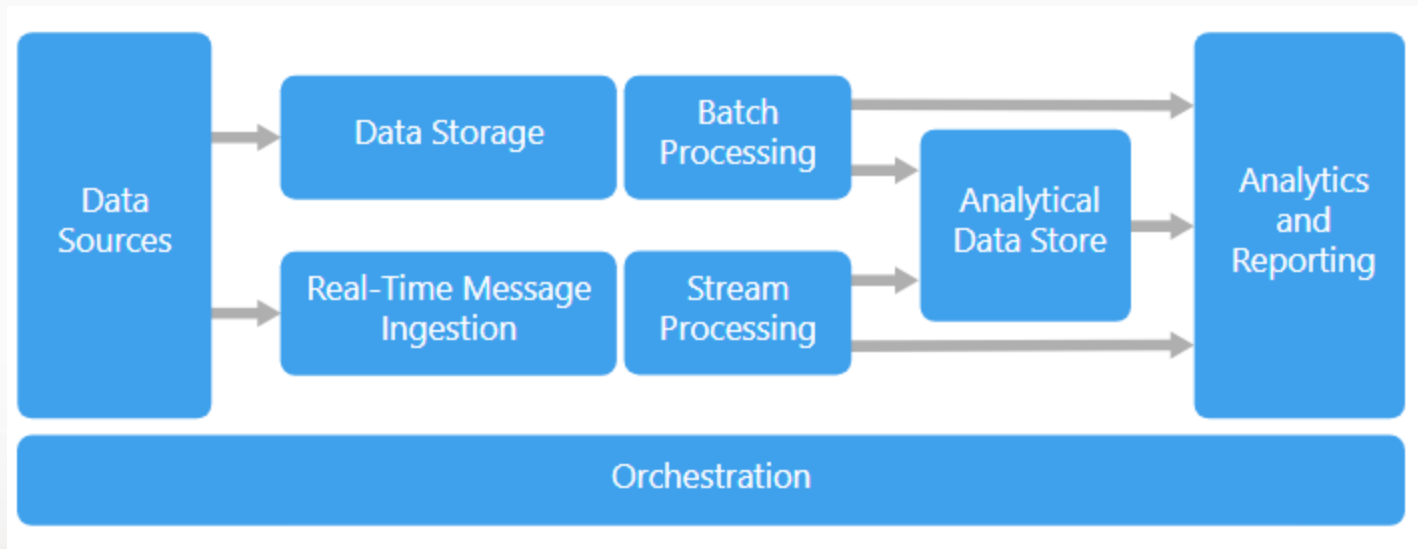
BIG-DATA

Uma arquitetura de big data é projetada para lidar com entrada, processamento e análise de dados que são muito grandes ou complexos para sistemas de bancos de dados tradicionais.

As soluções de big data geralmente envolvem um ou mais dos seguintes tipos de carga de trabalho:

- ✓ Processamento em lote de fontes de big data em repouso.
- ✓ Processamento em tempo real de big data em movimento.
- ✓ Exploração interativa de big data.
- ✓ Análise preditiva e aprendizado de máquina.

Engenharia de Software III



Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Origens de dados : todas as soluções de Big Data começam com uma ou mais fontes de dados. Exemplos incluem:

Armazenamento de dados de aplicativos, como bancos de dados relacionais.

Arquivos estáticos produzidos por aplicativos, como arquivos de log do servidor da web.

Origens de dados em tempo real, como dispositivos IoT.

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Armazenamento de dados : os dados para operações de processamento em lote normalmente são armazenados em um armazenamento de arquivos distribuído que pode conter grandes volumes de arquivos grandes em vários formatos. Esse tipo de loja é freqüentemente chamado de *LAKE STORE*.

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Processamento em lote : Como os conjuntos de dados são muito grandes, geralmente uma solução de big data deve processar arquivos de dados usando tarefas em lote de longa execução para filtrar, agregar e preparar os dados para análise. Normalmente, essas tarefas envolvem a leitura de arquivos de origem, o processamento e a gravação da saída em novos arquivos.

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Ingestão de mensagens em tempo real : se a solução incluir fontes em tempo real, a arquitetura deve incluir uma maneira de capturar e armazenar mensagens em tempo real para processamento de fluxo. Isso pode ser um armazenamento de dados simples, em que as mensagens recebidas são descartadas em uma pasta para processamento. No entanto, muitas soluções precisam de um armazenamento de entrada de mensagens para atuar como um buffer para mensagens e para suportar processamento de expansão, entrega confiável e outras semânticas de enfileiramento de mensagens.

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Processamento de fluxo : depois de capturar mensagens em tempo real, a solução deve processá-las filtrando, agregando e preparando os dados para análise. Os dados do fluxo processado são gravados em um coletor de saída.

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Armazenamento de dados analíticos : muitas soluções de Big Data preparam dados para análise e, em seguida, veiculam os dados processados em um formato estruturado que pode ser consultado usando ferramentas analíticas. O armazenamento de dados analíticos usado para atender a essas consultas pode ser um data warehouse relacional estilo Kimball, como visto na maioria das soluções tradicionais de business intelligence (BI).

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Análise e relatório : o objetivo da maioria das soluções de big data é fornecer insights sobre os dados por meio de análises e relatórios. Para capacitar os usuários a analisar os dados, a arquitetura pode incluir uma camada de modelagem de dados, como um cubo OLAP multidimensional ou um modelo de dados tabulares.

Engenharia de Software III

Componentes da arquitetura **BIG-DATA**

Orquestração : a maioria das soluções de big data consiste em operações de processamento de dados repetidas, encapsuladas em fluxos de trabalho, que transformam dados de origem, movem dados entre várias fontes e coletores, carregam os dados processados em um armazenamento de dados analítico ou enviam os resultados diretamente para um relatório ou painel .

Engenharia de Software III

TP 09, individual, pesquisa:

Quais as principais ferramentas que fornecem suporte para os componentes De BIG-DATA

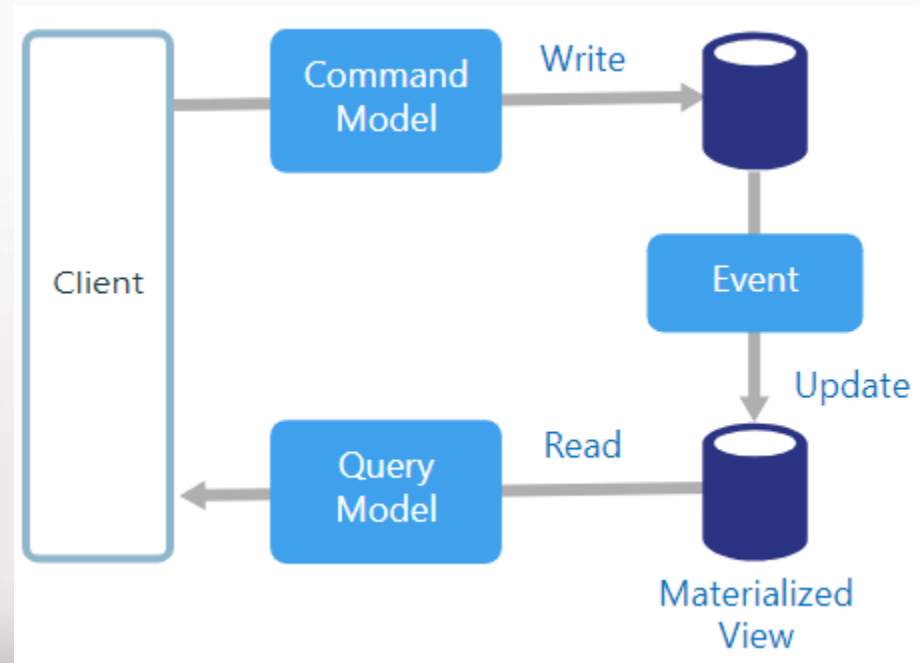
Incluir nome da ferramenta e descrição da ferramenta, para as seguintes empresas:

- Azure;
- AWS – Amazon Web servisse;
- Oracle;
- IBM;
- Google.

Engenharia de Software III

CQRS (Command and Query Responsibility Segregation, segregação por responsabilidade de comando de consulta)

É estilo de arquitetura que separa operações de leitura de operações de gravação.



Fonte: microsoft.com

Engenharia de Software III

Nas arquiteturas tradicionais, o mesmo modelo de dados é usado para consultar e atualizar um banco de dados. É simples e funciona bem para operações CRUD básicas. Em aplicativos mais complexos, no entanto, essa abordagem pode se tornar complicada. Por exemplo, no lado de leitura, o aplicativo pode executar muitas consultas diferentes, retornando objetos de transferência de dados (DTOs) com formas diferentes. O mapeamento de objetos pode se tornar complicado. No lado da gravação, o modelo pode implementar uma validação complexa e lógica de negócios.

Como resultado, você pode terminar com um modelo excessivamente complexo que faz coisas em excesso.

Outro problema potencial é que a leitura e a gravação de cargas de trabalho geralmente são assimétricas, com diferentes requisitos de desempenho e escalabilidade.

<https://docs.microsoft.com/pt-br/azure/architecture/guide/architecture-styles/cqrs>

Engenharia de Software III

O CQRS trata desses problemas, separando as leituras e gravações em modelos separados, usando **comandos** para atualizar dados e **consultas** para ler dados.

Os comandos devem ser baseados em tarefas, em vez de centrados nos dados.

Os comandos podem ser posicionados em uma fila para processamento assíncrono, em vez de ser processado de forma síncrona.

As consultas nunca modificam o banco de dados. Uma consulta retorna um DTO que não encapsula qualquer conhecimento de domínio.

Engenharia de Software III

Separação física dos bancos

Para maior isolamento, você pode separar fisicamente os dados de leitura de dados de gravação.

Nesse caso, o banco de dados de leitura pode usar seu próprio esquema de dados, otimizado para consultas. Por exemplo, ele pode armazenar uma view (visão pré-preenchidas sobre os dados em um ou mais armazenamentos de dados quando os dados não estiverem idealmente formatados para as operações de consulta necessárias). Isso pode ajudar a suportar consultas eficientes e extração de dados e melhorar o desempenho do aplicativo.

Engenharia de Software III

Separação física dos bancos

Se os bancos de dados de leitura e gravação separados forem usados, deverão ser mantidos em sincronia.

Normalmente, isso é feito com a publicação de um evento pelo modelo de gravação sempre que ele atualiza o banco de dados de publicação.

A atualização do banco de dados e a publicação do evento devem ocorrer em uma única transação.

Ele ainda pode usar um tipo diferente de armazenamento de dados. Por exemplo, o banco de dados de gravação pode ser relacional, enquanto o banco de dados de leitura é um banco de dados de documento.

Engenharia de Software III

Quando usar essa arquitetura?

- ✓ Domínios colaborativos em que muitos usuários acessem os mesmos dados, especialmente quando as cargas de trabalho de leitura e gravação forem assimétricas.
- ✓ O CQRS não é uma arquitetura de nível superior que se aplica ao sistema inteiro.
- ✓ Aplique o CQRS somente a esses subsistemas onde houver um valor claro na separação de leituras e gravações.

Engenharia de Software III

Benefícios

- ✓ **Dimensionamento independente.** O CQRS permite que as cargas de trabalho de leitura e gravação sejam dimensionadas de forma independente e pode resultar em menos contenções de bloqueio.
- ✓ **Esquemas de dados otimizados.** O lado de leitura pode usar um esquema que é otimizado para consultas, enquanto o lado de gravação usa um esquema que é otimizado para atualizações.
- ✓ **Segurança.** É mais fácil garantir que apenas as entidades do direito de domínio estejam executando gravações nos dados.

Engenharia de Software III

Benefícios

- ✓ **Separação de preocupações.** Isolar os lados de leitura e gravação pode resultar em modelos mais flexíveis e sustentáveis. A maior parte da lógica de negócios complexa vai para o modelo de gravação. O modelo de leitura pode ser relativamente simples.
- ✓ **Consultas mais simples.** Ao armazenar uma exibição materializada no banco de dados de leitura, o aplicativo poderá evitar junções complexas durante as consultas.

Engenharia de Software III

Desafios

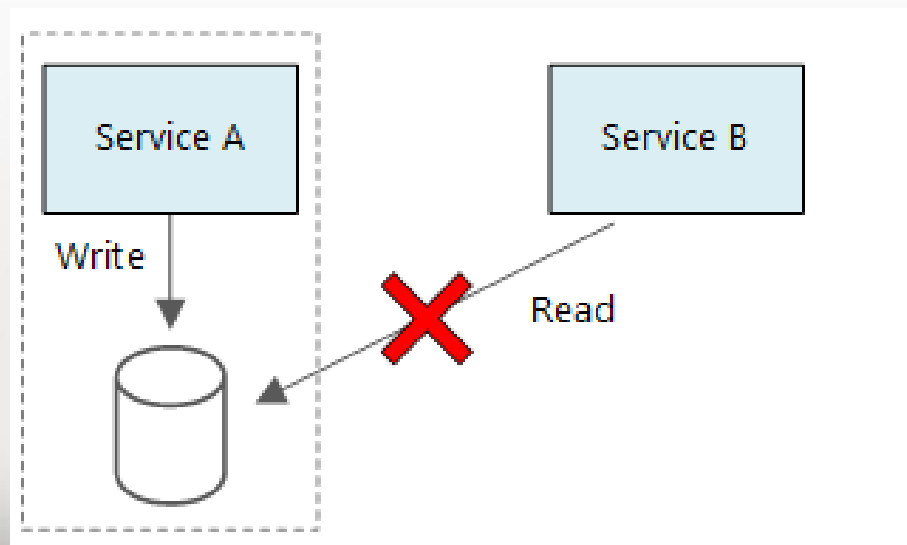
- ❖ **Complexidade.** A ideia básica do CQRS é simples. Mas isso poderá resultar em um design de aplicativo mais complexo, especialmente se eles incluírem o ***padrão Fornecimento de Eventos***.
- ❖ **Mensagens.** Embora o CQRS não necessite de mensagens, é comum usar mensagens para comandos de processo e publicar eventos de atualização. Neste caso, o aplicativo deve tratar as falhas de mensagem ou as mensagens duplicadas.
- ❖ **Consistência eventual.** Se você separar os bancos de dados de leitura e de gravação, os dados de leitura poderão ficar obsoletos.

Engenharia de Software III

CQRS em microserviços

O CQRS pode ser especialmente útil em uma arquitetura de microserviços.

Um dos princípios de microserviços é que um serviço não pode acessar diretamente o armazenamento de dados do outro serviço.



Engenharia de Software III

No diagrama a seguir, o Serviço A grava em um armazenamento de dados e o Serviço B mantém uma exibição materializada dos dados. Um serviço publica um evento sempre que ele grava no armazenamento de dados. O Serviço B assina o evento.

